Files and streams

CSC103 Programming Fundamentals / Lecture 28

Dr. Muhammad Farhan

Associate Professor of Computer Science
Department of Computer Science
COMSATS University Islamabad
Sahiwal Campus

Fall 2026

The problem for this lecture

Create a File object for data/marks.txt. Display its absolute path, existence and whether it is a regular file. Explain environment-dependent output.

Represent data/marks.txt using a File object.

Learning objectives

- Distinguish files, paths and streams
- Inspect file metadata
- Resolve relative paths against a working directory

CLO-3
Files and paths
The File class
Byte and character streams
File operations and failures

Files and paths

- A file stores persistent data outside the running program.
- A path identifies a location in a filesystem.
- Relative paths depend on the process working directory.

Example

```
marks.txt
data/marks.txt

Both are relative paths.
```

The File class

- java.io.File represents a pathname.
- Constructing a File object does not create the physical file.
- Methods such as exists and isFile inspect current metadata.

Example

```
java.io.File f = new java.io.File("marks.txt");
boolean present = f.exists();
```

Byte and character streams

- Byte streams move bytes. Character streams decode or encode text.
- Text encoding connects stored bytes to characters.
- Opening a stream acquires a resource that needs closing.

Example

```
Text: marks and names in UTF-8
Binary: an image or audio file

Use APIs appropriate to the data.
```

File operations and failures

- A missing file, a directory path and insufficient permission are different conditions.
- Reading and writing have different effects.
- Opening output can overwrite existing content.

Example

```
Before writing:
Choose a dedicated output path.
Decide overwrite or append behaviour.
Handle the opening failure.
```

An absolute path explains where Java looks

- A relative pathname resolves against the process working directory.
- Printing an absolute form is a useful diagnosis for a missing-file report.
- The same source program can resolve a relative name differently under another launcher.

Example

```
Relative: data/marks.txt
Working directory: a course practice folder
Resolved location: that folder plus data/marks.txt
```

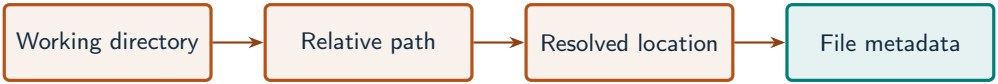
Metadata is an observation at one moment

- exists reports whether the path exists when the call checks it.
- isFile distinguishes an ordinary file from a directory.
- A later operation still needs error handling because state or permissions can change.

Example

```
exists=false does not create a file.
exists=true does not guarantee that a later read succeeds.
```

A path is interpreted from a working directory



What to notice

Constructing a File describes a path; it does not create the referenced file.

Key distinctions

File method	Question	Result type
getName()	Final pathname component?	String
getAbsolutePath()	Where does this path resolve?	String
exists()	Does the path currently exist?	boolean
isFile()	Does it identify a regular file?	boolean

These distinctions determine which operation or control structure fits the problem.

First example: execution

```
java.io.File file = new java.io.File("marks.txt");
System.out.println(file.getName());
System.out.println(file.isAbsolute());
```

First example: result

marks.txt
false
No physical file is created by the constructor.
The relative path will resolve in the program working directory.

Byte and character streams
File operations and failures

Guided practice

Create a File object for data/marks.txt. Display its absolute path, existence and whether it is a regular file. Explain environment-dependent output.

Attempt the solution before continuing.
Use the definitions and examples from this lecture.

Solution strategy

- Represent data/marks.txt using a File object.
- Print the absolute path to expose working-directory resolution.
- Inspect existence and file type without creating or modifying the path.

The next program uses fixed inputs so its result can be reproduced.

Complete solution

```
public class Solution28 {
  public static void main(String[] args) throws Exception {
    java.io.File file = new java.io.File("data/marks.txt");
    System.out.println(file.getAbsolutePath());
    System.out.println(file.exists());
    System.out.println(file.isFile());
  }
}
```

Execution trace

Observation	In an empty practice folder	Reason
Absolute path	Working folder + data/marks.txt	Relative path resolution
exists()	false	No file created
isFile()	false	Missing path is not a regular file

Follow the changing state in execution order.

Solution result

```
[working directory]\data\marks.txt
false
false
```

The absolute path and metadata depend on the working directory and existing files. This example assumes an empty practice folder.

A common incorrect approach

new java.io.File("report.txt");
// Student expects an empty report.txt to appear.

Example

```
The constructor only represents a pathname. A file-creation or writing operation is needed to
    create the physical file.
```

Boundary and failure checks

Try the program with a missing path, an existing text file and an existing directory.

Use getAbsolutePath(), exists() and isFile(). The exact path and Boolean results depend on where the program runs and what exists there.

Check your understanding

Does a relative path start at the source-file folder automatically? What closes a stream reliably?

Write a prediction and explain the rule that supports it.

Answer and explanation

No, it starts from the working directory. A try-with-resources statement closes an AutoCloseable resource on normal and exceptional exit.

The constructor only represents a pathname. A file-creation or writing operation is needed to create the physical file.

Compare cases and predict outcomes

Operation	Creates a file?	What it provides
new File("marks.txt")	No	Path object
getAbsolutePath()	No	Absolute path text
exists()	No	Existence observation

Discuss

Explain each expected result before running the example. Which case would reveal a wrong assumption?

Apply the idea

Independent practice

Explain why new File("marks.txt") followed by exists() may be false. Show how to print the resolved path for diagnosis.

1. Work through the input by hand.
2. Write or correct the program.
3. Justify the expected result.

Solution and reasoning

```
java.io.File file = new java.io.File("marks.txt");
System.out.println(file.getAbsolutePath());
System.out.println(file.exists());
```

- A path object does not imply that storage exists at that location.
- The working directory can differ between a terminal and an IDE.
- The printed path and Boolean depend on the actual run environment.

Creating a file is different from naming it

- Constructing File creates a path object; createNewFile attempts to create storage.
- createNewFile returns false if the named file already exists.
- Creation may throw IOException; an existence check does not guarantee later creation will succeed.

Source: supplied ISB campus slides. See speaker notes and ISB_Source_Map.md.

Worked problem: Creating a file is different from naming it

Task

Create a private temporary directory and call createNewFile twice on the same filename.

Input: Values are fixed in the program for a reproducible demonstration.

Before running

Predict the output and identify the variables that change.

Complete ISB example (1/1)

```
public class IsbLecture28 {
  public static void main(String[] args) throws Exception {
    java.nio.file.Path folder = java.nio.file.Files.createTempDirectory("csc103-isb-");
    java.io.File file = folder.resolve("first.txt").toFile();
    System.out.println(file.createNewFile());
    System.out.println(file.createNewFile());
    System.out.println(file.isFile());
  }

}
```

Worked trace and expected result

Operation	Return / observation	Explanation
First createNewFile	true	New file created
Second createNewFile	false	File already exists
isFile	true	Path names a regular file

Expected console output

```
true
false
true
```

Extend the ISB example

Practice

Why is an absolute path not universally wrong, despite the source warning against it?

Explain your reasoning

Absolute paths can be appropriate when explicitly supplied or configured. Hard-coded machine-specific paths reduce portability.

Lecture summary

- files, paths and streams
- Inspect file metadata
- relative paths against a working directory

- Represent data/marks.txt using a File object.
- Print the absolute path to expose working-directory resolution.
- Inspect existence and file type without creating or modifying the path.

After-class practice

Inspect a dedicated practice file and a directory using File methods. Record differences without deleting either.

Syllabus reading
Liang Ch 12
Next lecture: Reading and writing text files